

# John the Ripper

John the Ripper (JtR) isn't a vulnerability class like the last few guides — it's the **tool** that turns "we obtained a password hash" into "we obtained the password," so this guide is structured slightly differently: **Part 1 covers the core cracking modes** (the fundamental strategies for guessing a plaintext that hashes to a target value), **Part 2 covers advanced/format-specific techniques** (rules, masks, and handling the many hash formats you'll actually encounter). Worth locking in the distinction immediately: every mode in Part 1 answers *"what plaintexts do I try?"* — Part 2 is entirely about answering *"how do I try them smarter, and against what exact hash format?"* This connects directly back to your CRLF/log-forging guide's point about credential material ending up in logs, and to any SQLi/LFI guide where a dumped hash column is the actual payoff of the initial exploitation.

## 1. How Password Cracking Actually Works — The Foundation

A password hash is a **one-way function** — you can't mathematically reverse it back into the plaintext. Cracking doesn't reverse the hash; it **guesses a plaintext, hashes the guess with the same algorithm, and compares the result** to the target hash. Do this fast enough, against a good enough set of guesses, and eventually a guess's hash matches.

1. Attacker obtains a hash (dumped database, /etc/shadow, captured network hash like NTLMV2, extracted from a memory dump, etc.)
2. Attacker feeds JOHN a candidate plaintext
3. JOHN hashes the candidate using the SAME algorithm/parameters as the target hash (salt, iteration count, hash function)
4. If candidate\_hash == target\_hash → CRACKED, plaintext recovered
5. If not → try the NEXT candidate, repeat at massive scale

**The single fact that governs every mode below — cracking speed is entirely a function of (a) how expensive the hash algorithm is to compute per guess, and (b) how well-targeted your candidate list is.** A fast, unsalted hash (raw MD5) can be tried billions of times per second on modern hardware; a deliberately slow, memory-hard hash (bcrypt, Argon2) might allow only thousands of guesses per second on the same hardware — this single variable

is the entire reason password storage design (Section 15) matters as much as it does.

Fast hashes (crackable at huge scale) → MD5, SHA-1, SHA-256, NTLM

Slow/hardened hashes (crackable slowly) → bcrypt, scrypt, Argon2, PBKDF2 with high iterations

## PART 1: Core Cracking Modes

### 2. Single Crack Mode

The fastest mode to try first — JtR generates candidate guesses **from context it already has**, primarily the username and any GECOS/full-name field associated with the account, applying mangling rules on top:

```
john --single --format=raw-md5 hashes.txt
```

```
Username: jsmith → candidates tried include:  
jsmith, Jsmith, JSMITH, jsmith1, jsmith123, jsmith!, ...
```

This exploits the extremely common real-world pattern of users basing passwords on their own username, name, or account details — a shockingly high hit rate for essentially zero cost, which is exactly why it's the recommended **first** mode to run against any new hash dump before reaching for anything heavier.

### 3. Wordlist (Dictionary) Mode

The workhorse mode — JtR hashes every entry in a supplied wordlist file and compares each to the target:

```
john --wordlist=rockyou.txt --format=raw-md5 hashes.txt
```

```
rockyou.txt (the canonical, most-used wordlist - ~14 million real,  
breach-derived passwords) contains lines like:
```

```
123456  
password  
iloveyou
```

```
letmein
...
```

Effectiveness here is almost entirely a function of **wordlist quality**, not JtR itself — a targeted, breach-relevant wordlist (rockyou.txt, or a list built from previously-breached credentials of the SAME organization) will vastly outperform a generic dictionary, since real humans reuse passwords across services constantly.

## 4. Rule-Based Wordlist Mode

Combining a wordlist with **mangling rules** multiplies its effective coverage without needing a bigger list — rules apply systematic transformations (capitalization, leetspeak substitution, appended digits/symbols) to every wordlist entry:

```
john --wordlist=rockyou.txt --rules=best64 --format=raw-md5 hashes.txt
```

```
Base word: password
Rules generate: Password, password1, p@ssword, PASSWORD123,
                drowssap (reversed), Password!, ...
```

This is covered in more depth in Section 8, but it's worth flagging here as the natural evolution of Section 3 — most real-world cracking sessions run wordlist+rules together as the default, rather than a bare wordlist alone.

## 5. Incremental (Brute-Force) Mode

Tries **every possible character combination** within a defined character set and length range — exhaustive, guaranteed to eventually find any password within the search space, but exponentially slower as length/charset grows:

```
john --incremental=Digits --format=raw-md5 hashes.txt
john --incremental=Alnum --format=raw-md5 hashes.txt
```

```
Search space size = (charset size) ^ (password length)

4-digit PIN (10 chars, len 4)    → 10,000 combinations - trivial
```

8-char alphanumeric (62 chars) → ~218 TRILLION combinations - infeasible on a single machine against a slow hash

**When this mode is actually viable:** short/constrained targets (numeric PINs, short fixed-length tokens) or as a **last resort** after wordlist+rules has been exhausted against a fast hash — rarely the first mode reached for, given the search space explosion.

## 6. External / Custom Mode

Allows a user-supplied program (written in a restricted C-like syntax within JtR's config) to generate candidates according to fully custom logic — used when the target's password policy or generation pattern is known and doesn't fit any built-in mode:

Example use case: you know the organization enforces passwords of the exact form CompanyName + 2-digit-year + special-char (e.g. Acme26!) - an external mode generator can be written to produce EXACTLY that pattern space, far more efficiently than brute-forcing or hoping a wordlist happens to contain it

# PART 2: Advanced & Format-Specific Techniques

## 7. Identifying the Hash Format First

Before any mode runs correctly, JtR needs to know **which hashing algorithm and format** it's looking at — guessing wrong wastes the entire cracking session on a mismatched comparison function:

```
john --list=formats          # list every supported format
john --format=nt hashes.txt  # explicitly specify Windows NTLM
```

Common formats and their typical hash appearance:

|             |                                                               |
|-------------|---------------------------------------------------------------|
| raw-md5     | → 32 hex characters, no salt visible                          |
| raw-sha256  | → 64 hex characters                                           |
| nt          | → Windows NTLM (from SAM/NTDS dumps)                          |
| sha512crypt | → \$6\$salt\$hash (common on modern Linux /etc/shadow)        |
| bcrypt      | → \$2a\$ / \$2b\$ / \$2y\$ prefix + cost factor + salt + hash |

```
krb5tgs    → Kerberos ticket hash (for Kerberoasting attacks)
netntlmv2   → captured over-the-network Windows auth hash
```

- `-format` mismatches are the single most common reason a cracking session silently produces zero results despite a correct wordlist — always confirm the format before assuming the wordlist/rules are at fault.

## 8. Rule Syntax — Writing Custom Mangling Rules

Beyond the built-in `best64` / `jumbo` rule sets referenced in Section 4, custom rules can be defined in `john.conf` using JtR's compact rule syntax:

```
[List.Rules:Custom]
c $1 $2 $3          # Capitalize, then append 1, 2, 3
sa@                 # substitute 'a' with '@'
so0                 # substitute 'o' with '0'
Az"!"               # append "!" at the end
```

```
john --wordlist=custom.txt --rules=Custom hashes.txt
```

Writing targeted custom rules based on a **known organizational password policy** (minimum length, required special character, common corporate naming conventions observed in a previous breach of the SAME org) consistently outperforms generic built-in rule sets in real engagements.

## 9. Mask Attack Mode

A more efficient, targeted variant of brute-force (Section 5) — instead of trying every character at every position, a mask defines **which character class goes in which position**, dramatically shrinking the search space when partial structure is known or suspected:

```
john --mask='?u?l?l?l?l?d?d?d?d' --format=raw-md5 hashes.txt
```

```
?u = uppercase letter    ?l = lowercase letter
?d = digit                ?s = special character
```

```
Mask ?u?l?l?l?l?d?d?d?d matches the pattern:
  Uppercase + 4 lowercase + 4 digits
```

e.g. "Snake2024", "Tiger1987" - a VERY common human password structure (Capitalized word + 4-digit year)

This is frequently the single highest-value technique against a corpus of human-chosen passwords, since it encodes the *structural* pattern humans actually follow rather than trying to enumerate every possible string blindly.

## 10. Hybrid Mode — Wordlist + Mask Combined

Appends or prepends a mask-generated suffix/prefix onto every wordlist entry, combining Section 3's real-word coverage with Section 9's structural targeting:

```
john --wordlist=rockyou.txt --mask='?w?d?d?d?d' --format=raw-md5 hashes.txt
```

?w in the mask represents the wordlist word itself:  
rockyou.txt word "dragon" + mask ?d?d?d?d  
→ dragon0000, dragon0001, ... dragon9999

## 11. GPU Acceleration via John the Ripper Jumbo / OpenCL Formats

The "jumbo" community-extended fork of JtR adds GPU-accelerated formats for many algorithms, offering orders-of-magnitude speedup over CPU-only cracking for compatible hash types:

```
john --format=nt-opencl --wordlist=rockyou.txt hashes.txt
```

Fast hashes (NTLM, raw MD5/SHA) see the LARGEST GPU speedup - often 100x+ over CPU

Deliberately slow/memory-hard hashes (bcrypt, Argon2) see MUCH smaller relative gains, since they're specifically engineered to resist exactly this kind of parallelization

Worth noting **hashcat** is generally considered the more mature/faster tool specifically for GPU cracking today — JtR's strength remains its enormous format coverage, CPU-mode flexibility, and rule engine, so many practitioners use both: JtR for format identification/odd formats/single-crack, hashcat for large-scale GPU wordlist+mask runs.

## 12. Session Management for Long-Running Cracks

Cracking sessions against large hash dumps or wide search spaces can run for hours/days — JtR supports pausing and resuming without losing progress:

```
john --session=engagement1 --wordlist=rockyou.txt hashes.txt
# Ctrl+C to pause
john --restore=engagement1
john --show hashes.txt          # display all cracked results so far
```

## 13. Testing / Engagement Methodology

1. Obtain the hash dump and IDENTIFY the format precisely (Section 7) - never guess, confirm via hash structure/prefix or the source system's documented algorithm
2. Run --single mode FIRST (Section 2) - fast, frequently surprisingly effective, essentially free
3. Run wordlist + best64/jumbo rules (Sections 3-4) against a breach-relevant wordlist - the highest-value general-purpose step for most engagements
4. If organizational password policy is known/suspected, build and run a targeted mask (Section 9) or hybrid attack (Section 10)
5. Reserve pure incremental/brute-force (Section 5) for short, constrained targets only, given the exponential cost
6. Track and report which passwords were cracked, via WHICH method, and how quickly - this data itself is valuable engagement output, demonstrating real-world password policy weaknesses to the client far more concretely than a generic "enforce complexity requirements" recommendation

## 14. Legal & Ethical Note

Password cracking against hashes you do not have explicit, documented authorization to test (a signed engagement scope, your own lab environment, or a hash you legitimately own) is illegal in most jurisdictions, regardless of technique sophistication. Every example in this guide assumes an authorized penetration test, CTF/lab environment, or your own credential recovery.

## 15. Prevention — Defending Against Password Cracking

- Use a deliberately SLOW, memory-hard hashing algorithm for password storage - bcrypt, scrypt, or Argon2id - never a fast general-purpose hash (MD5, SHA-1, SHA-256) for passwords, even salted, since fast hashes remain crackable at enormous scale regardless of salting
- ALWAYS salt - a per-user random salt defeats precomputed rainbow tables entirely and forces an attacker to crack each hash individually rather than once for the whole dump
- Enforce meaningful minimum length over complex character-class requirements - length increases the incremental/mask search space exponentially, while forced complexity rules (Section 8-9) often just push users toward PREDICTABLE patterns (Capital + word + year + symbol) that a well-built mask attack targets directly
- Screen new passwords against known-breached password lists (the same rockyou.txt-style corpora attackers use) at signup/change time, rejecting matches outright
- Implement account lockout / rate limiting on ONLINE authentication attempts - irrelevant to offline cracking of an already-dumped hash, but closes the parallel online brute-force path entirely
- Rotate and audit credentials immediately after any suspected breach - cracked passwords from an old dump remain valuable to attackers indefinitely if never rotated, especially given widespread password reuse across services

## 16. Practical Lab Example

TryHackMe and Hack The Box both have dedicated JtR-focused rooms/labs walking through format identification and mode selection against realistic dumped hash sets; PortSwigger's Web Security Academy doesn't cover offline cracking directly (it's outside their web-app-specific scope) but frequently sets up labs where the PAYOFF of a SQLi/LFI is a dumped hash column - exactly where JtR picks up.

Suggested practice order:

1. Grab a small raw-md5 hash set (or generate your own test hashes) and run --single, then wordlist+rules (Sections 2-4) - get comfortable with format flags and basic syntax first
2. Move to a sha512crypt or bcrypt sample set and observe the dramatic speed difference firsthand (Section 1's core lesson, made concrete)
3. Practice building a mask attack (Section 9) against a synthetic set of passwords you generate yourself following a known pattern (Capital + word + 4 digits) - confirm you can recover them FASTER via a targeted mask than via a bare wordlist



# Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

| Framework         | Mapping                                                                                                                                                                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CWE               | CWE-916 (Use of Password Hash With Insufficient Computational Effort - the storage-side root cause JtR exploits), CWE-521 (Weak Password Requirements, relevant to Section 15's policy guidance)         |
| OWASP Top 10:2025 | A02:2025 Security Misconfiguration / Cryptographic Failures territory (weak hashing algorithm choice, missing/reused salts)                                                                              |
| CVSS              | Not directly CVSS-scored itself (JtR is a tool, not a vulnerability) - severity is inherited from whatever exposed the hash dump in the first place (SQLi, LFI, misconfigured backup exposure, etc.)     |
| Cyber Kill Chain  | Actions on Objectives (cracking typically happens AFTER initial access/exfiltration has already occurred, converting a dumped hash into a usable credential for further lateral movement or persistence) |
| MITRE ATT&CK      | T1110.002 (Brute Force: Password Cracking - the exact technique this entire guide covers), T1003 (OS Credential Dumping, the typical PRECEDING step that produces the hash JtR then cracks)              |

## Quick Reference Cheat Sheet

### CORE MODES (what candidates to try):

```
--single                → context-derived guesses (username-based)
--wordlist=FILE          → dictionary attack
--wordlist=FILE --rules=SET → dictionary + mangling rules
--incremental=MODE       → exhaustive brute-force
--mask='?u?l?l?l?d?d?d' → structural, targeted brute-force
--wordlist=FILE --mask='?w?d?d' → hybrid (wordlist + mask suffix)
```

### ESSENTIAL FLAGS:

```
--format=X              → specify hash type (CHECK THIS FIRST always)
--list=formats           → list all supported formats
--session=NAME           → name a resumable session
--restore=NAME           → resume a paused session
--show FILE              → display all cracked results
```

### RECOMMENDED ORDER FOR A NEW HASH DUMP:

1. Identify format precisely (Section 7)
2. --single (fast, near-free)
3. wordlist + best64/jumbo rules (highest general value)
4. targeted mask/hybrid if policy pattern is known
5. incremental only as a last resort, on short/constrained targets

Root lesson: cracking speed is a function of (hash algorithm cost)  
x (candidate list quality) - a bcrypt hash against a weak wordlist

can be MORE resistant than a raw MD5 hash against a well-targeted mask, regardless of which tool or how much hardware you throw at it

---